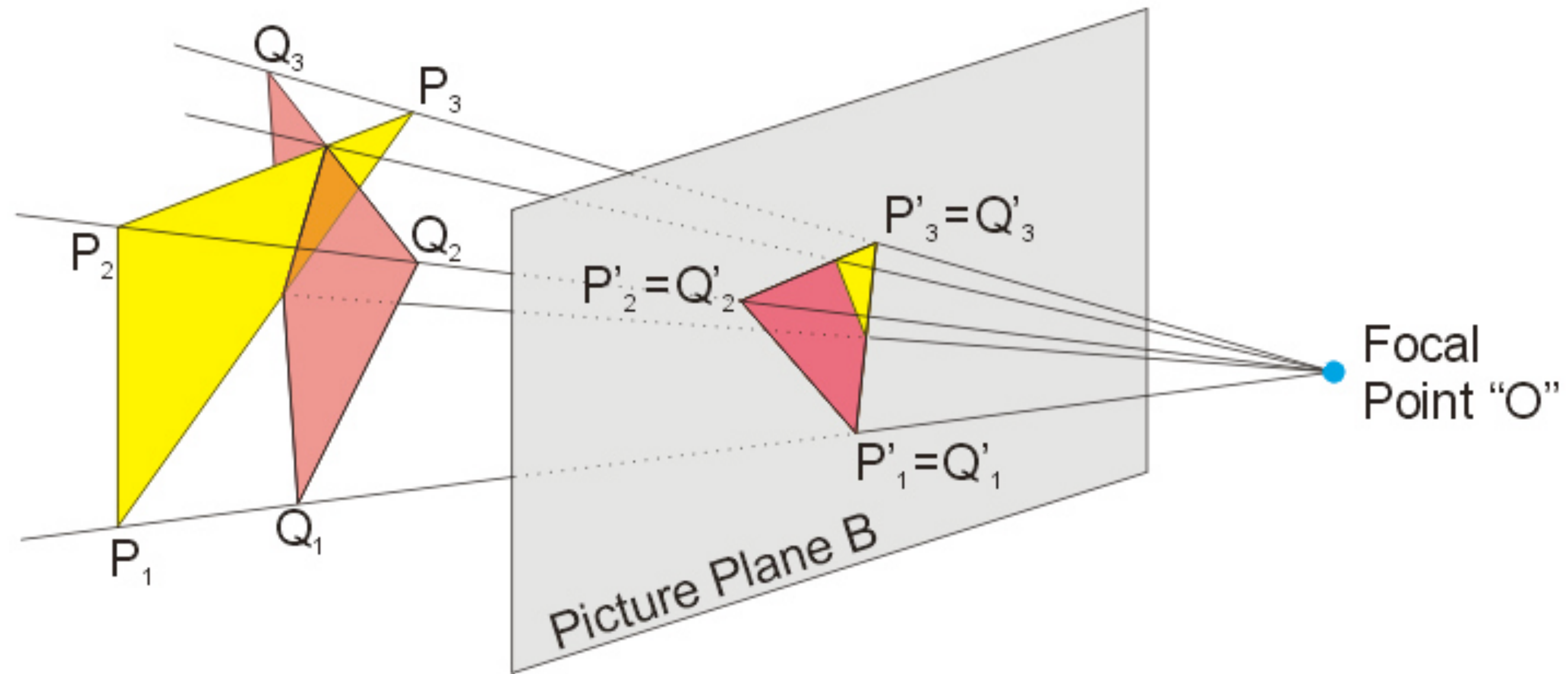
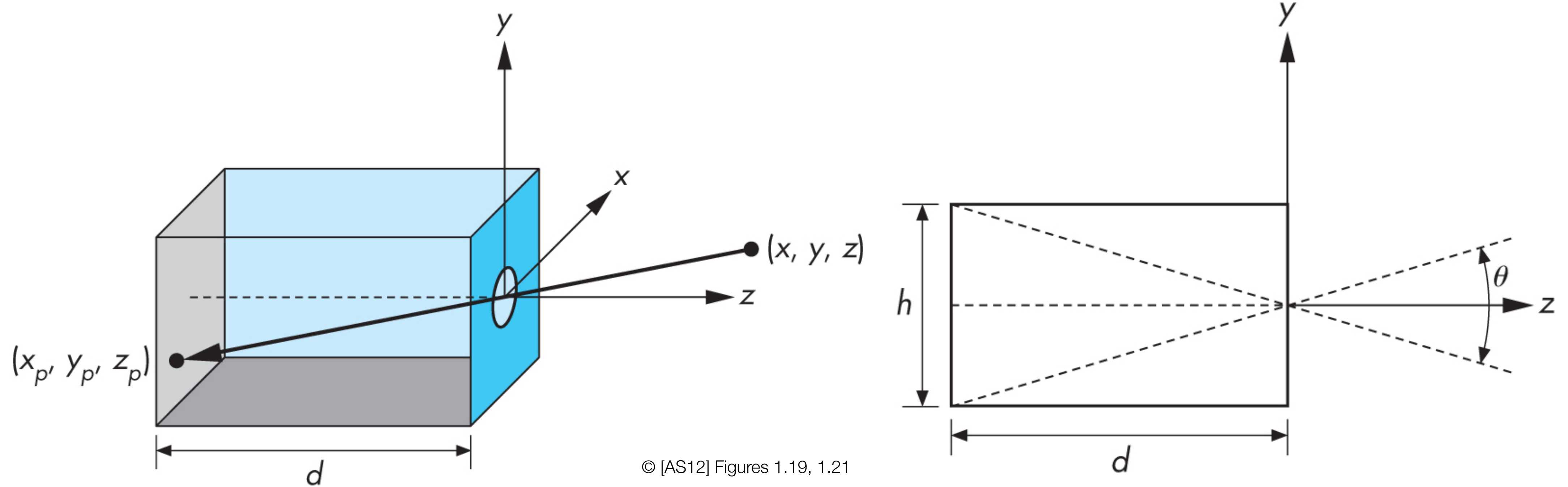


Perspective Projection

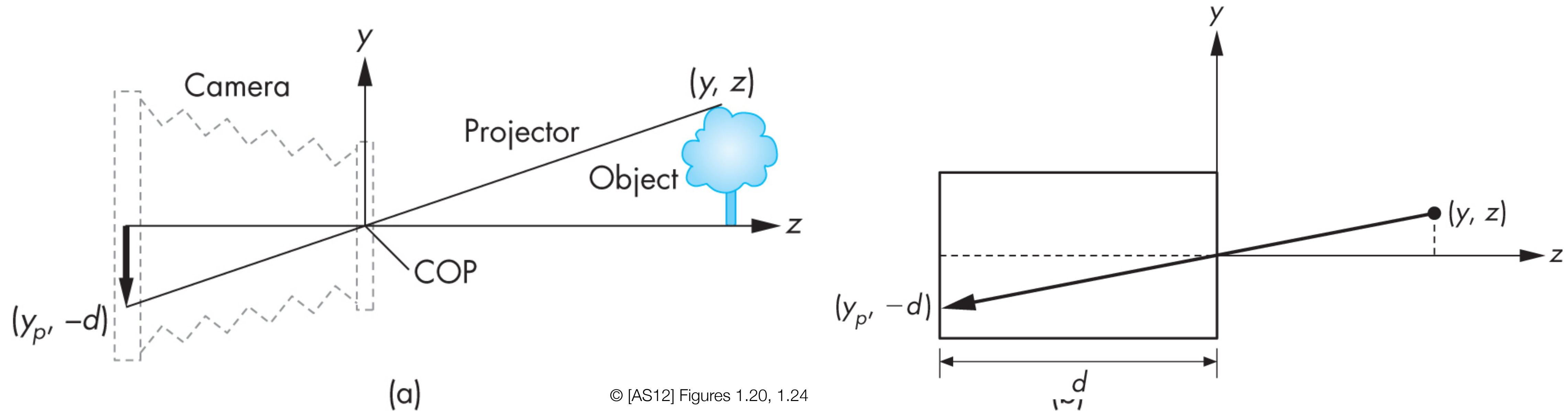


Pinhole Camera Review



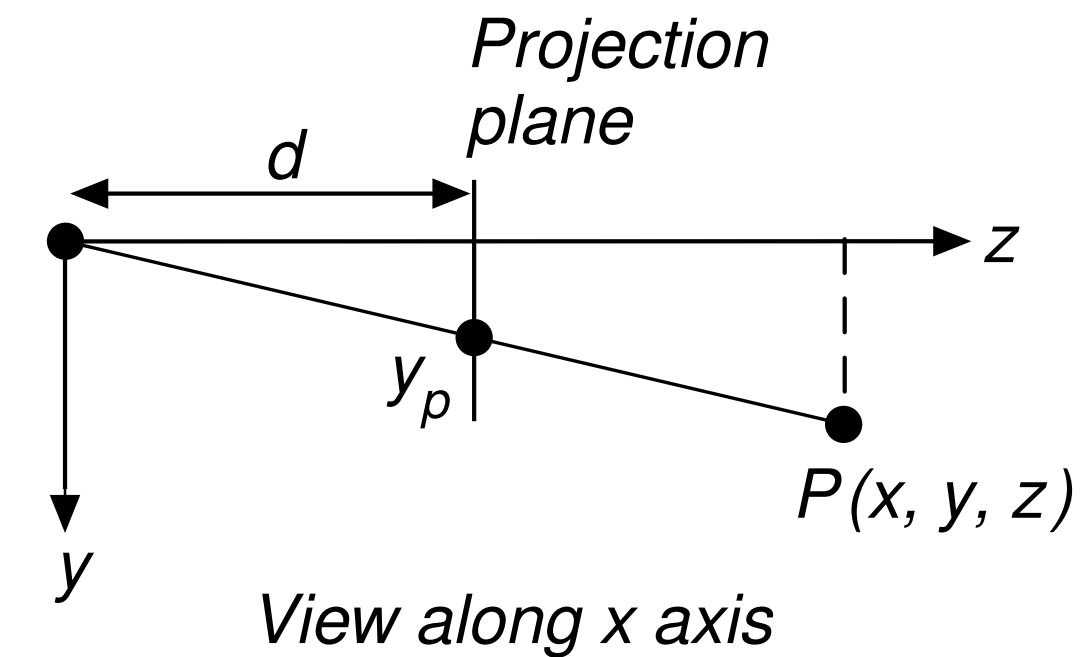
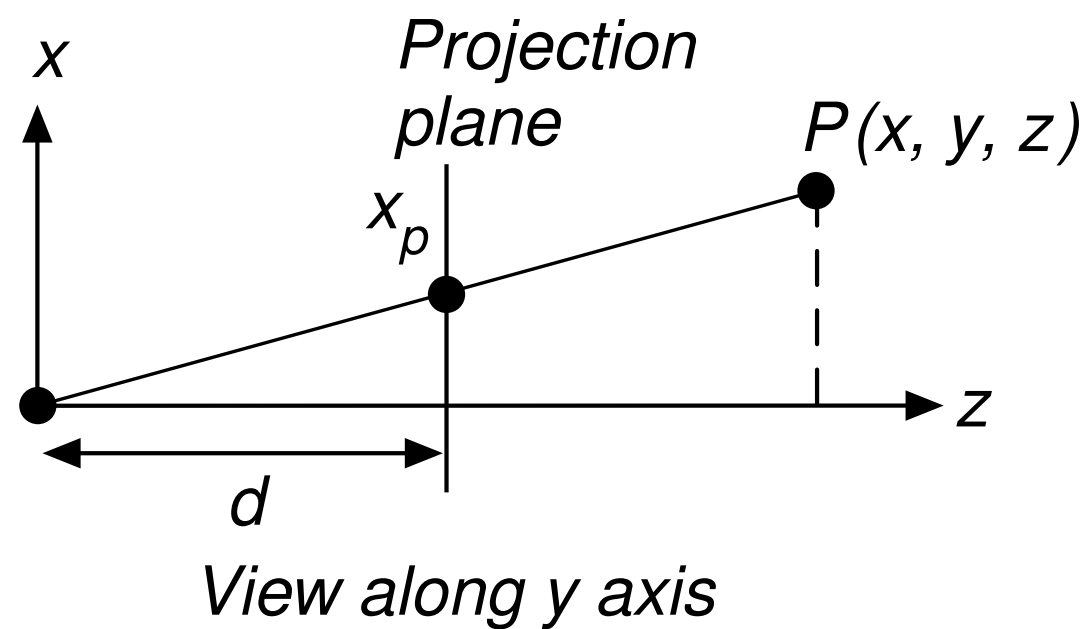
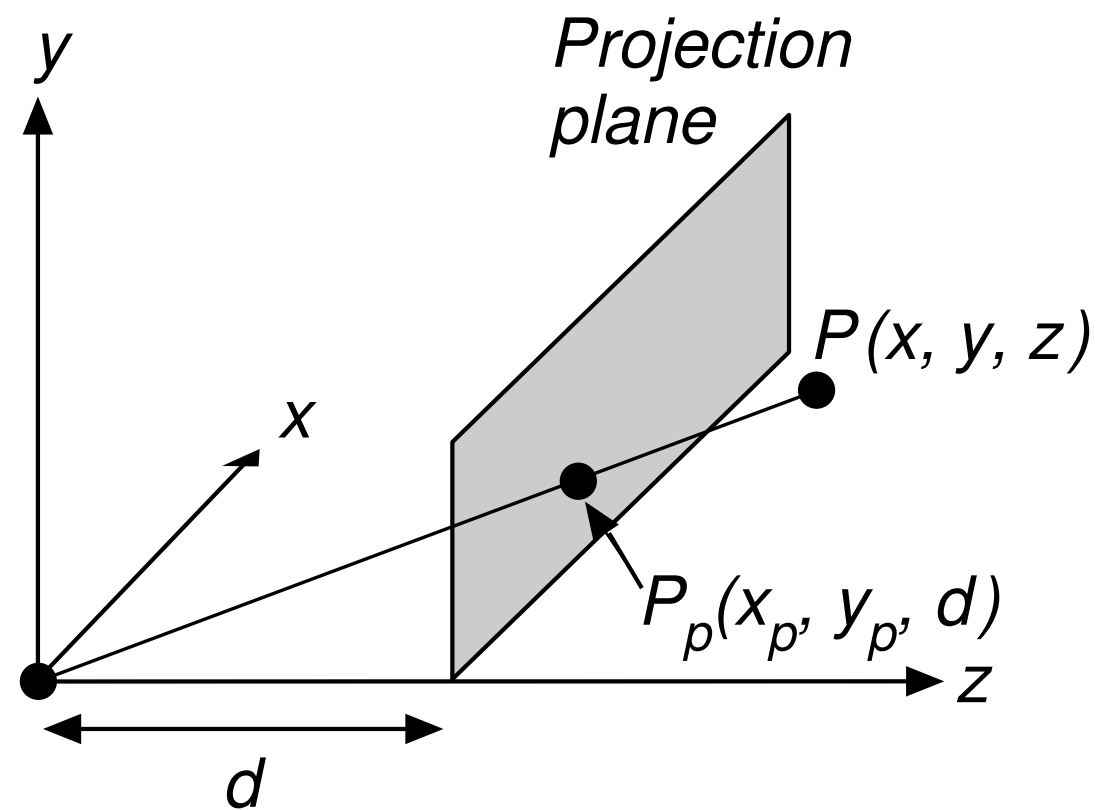
- Image formed on the back image plane of the box by rays entering through a small hole in the front
- Angle of view θ of view-frustum defined by the size h of the image plane and its focal length d

Virtual Projection Plane



- Projection onto image plane defined by simple planar trigonometry
 - ▶ infinite depth-of-field but admits almost no light through pinhole
- Image plane in virtual camera model can be anywhere
 - ▶ focal length $\pm d$ connected to angle of view θ and clip window size

Perspective Division



© [AW94] Figure 6.31

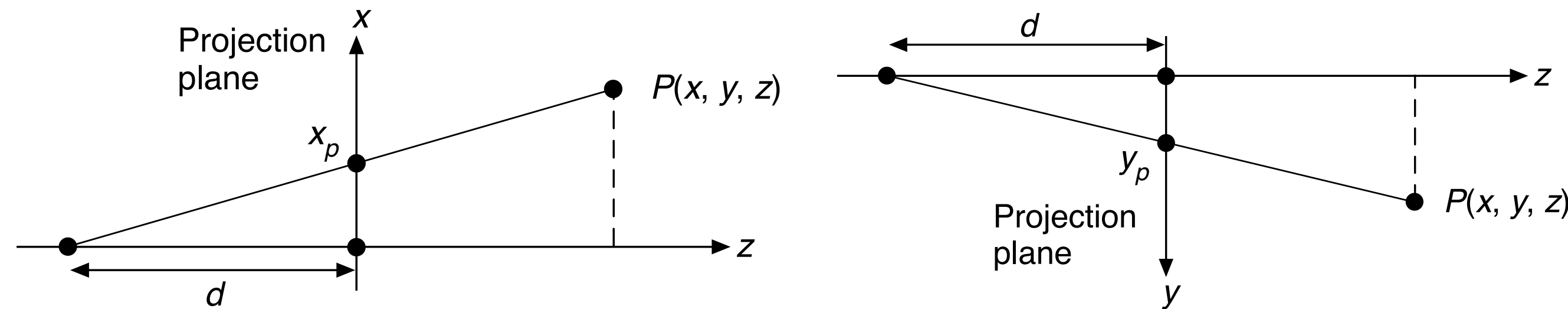
- Configuration with eye at origin, viewing along the z -axis and projection plane at $z = d$
 - ▶ distance d is focal length, a scaling factor for a given fixed angle of view
- Perspective projection: $P=(x,y,z) \rightarrow P_p=(x_p,y_p,d)$
 - ▶ focal $x_p/d = x/z$ and $y_p/d = y/z$ or $x_p = x \cdot d/z$ and $y_p = y \cdot d/z$
 - ▶ factor $/z$ is **perspective division** which is responsible for the diminishing size with distance

Matrix Representation

$$\begin{bmatrix} X \\ Y \\ Z \\ W \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1/d & 0 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \\ z/d \end{bmatrix} \quad \left(\frac{X}{W}, \frac{Y}{W}, \frac{Z}{W} \right) = (x_p, y_p, z_p) = \left(\frac{x}{z/d}, \frac{y}{z/d}, d \right)$$

- Perspective projection $P_p = M_{\text{per}} \cdot P$ can be represented by a 4x4 homogeneous transformation matrix M_{per}
- Transformation of $(x, y, z, 1)^T$ by M_{per} yields homogeneous point $(X, Y, Z, W)^T$
 - ▶ conversion from homogeneous space to 3D Cartesian space done by division through W
 - yields expected point on viewing plane at distance d

More Projection Matrices



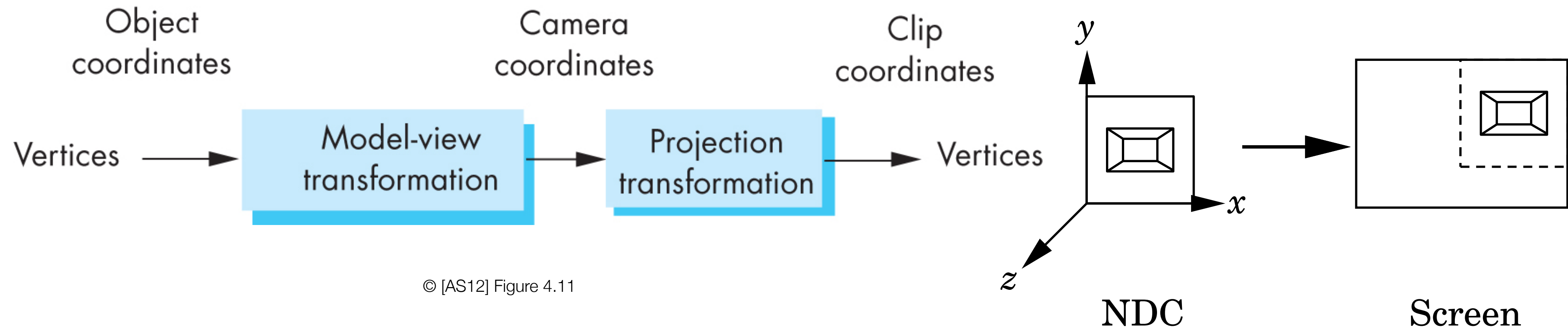
© [AW94] Figure 6.32

$$M_{\text{per}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1/d & 1 \end{bmatrix}$$

- Viewing plane at origin $z = 0$ and COP behind image plane at $z = -d$ results in alternative projection matrix M'_{per}
- Orthographic parallel projection onto $z = 0$ given by M_{ort}
 - ▶ simply dropping the z-coordinate
- General projection matrices M_{gen} exist that capture all configurations

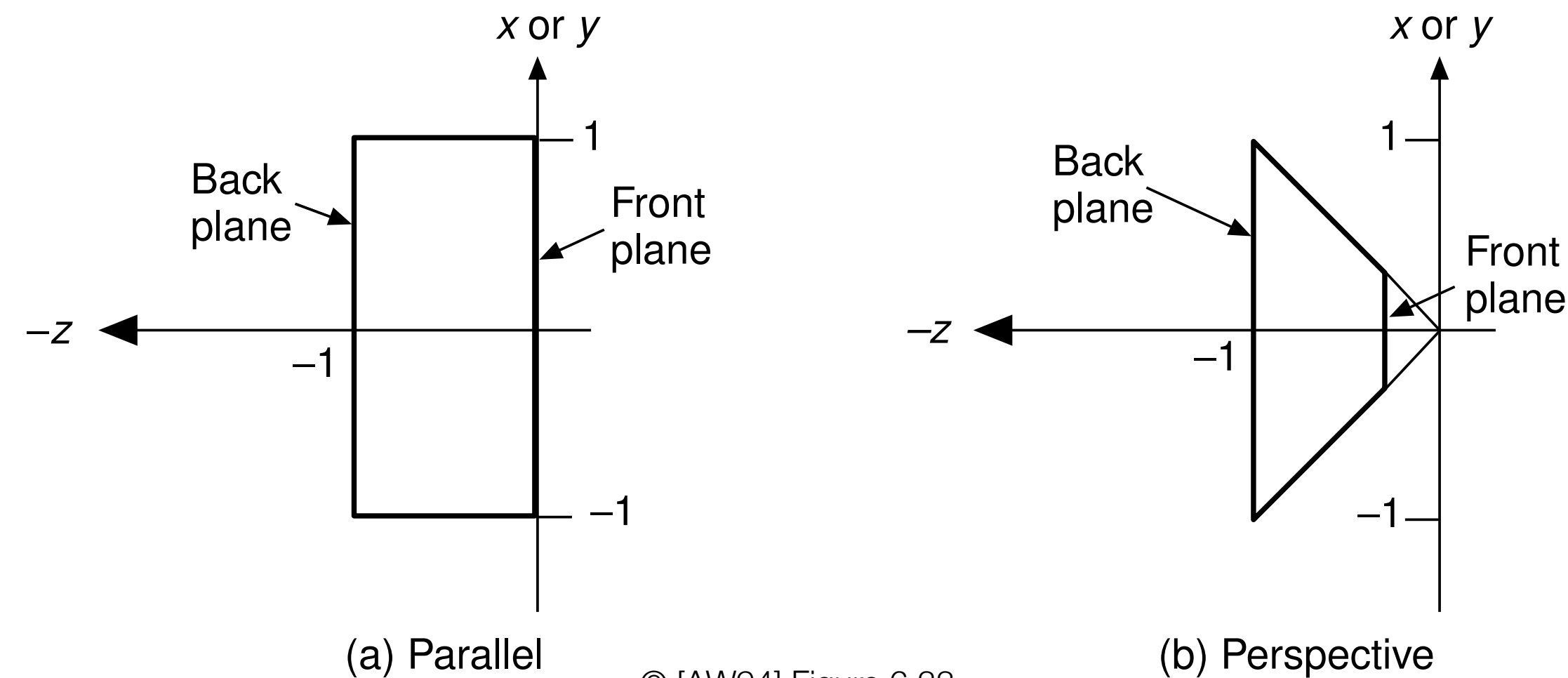
$$M_{\text{ort}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Viewing Implementation



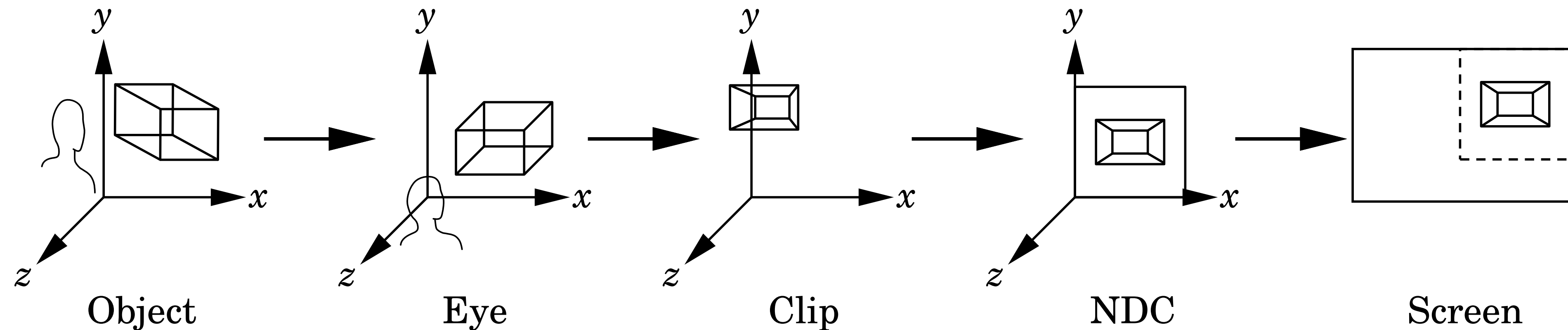
- Model-view transform, project-clip, viewport transform sequence
 - ▶ can combine projection and viewport-transform into one
- Find normalized transformations N_{per} (N_{par}) to transform arbitrary perspective (or parallel) view volume into canonical form

Canonical View Volume



- Canonical view volume defined as normalized box
 - ▶ canonical parallel-projection view volume $[-1,1] \times [-1,1] \times [-1,0]$
 - ▶ canonical perspective-projection view volume given by clipping planes
$$x = z, \quad x = -z, \quad y = z, \quad y = -z, \quad z = -z_{min}, \quad z = -1$$
- Goal: transform any camera and projection setup into canonical view volume
 - ▶ clipping in this transformed clip-space is easier

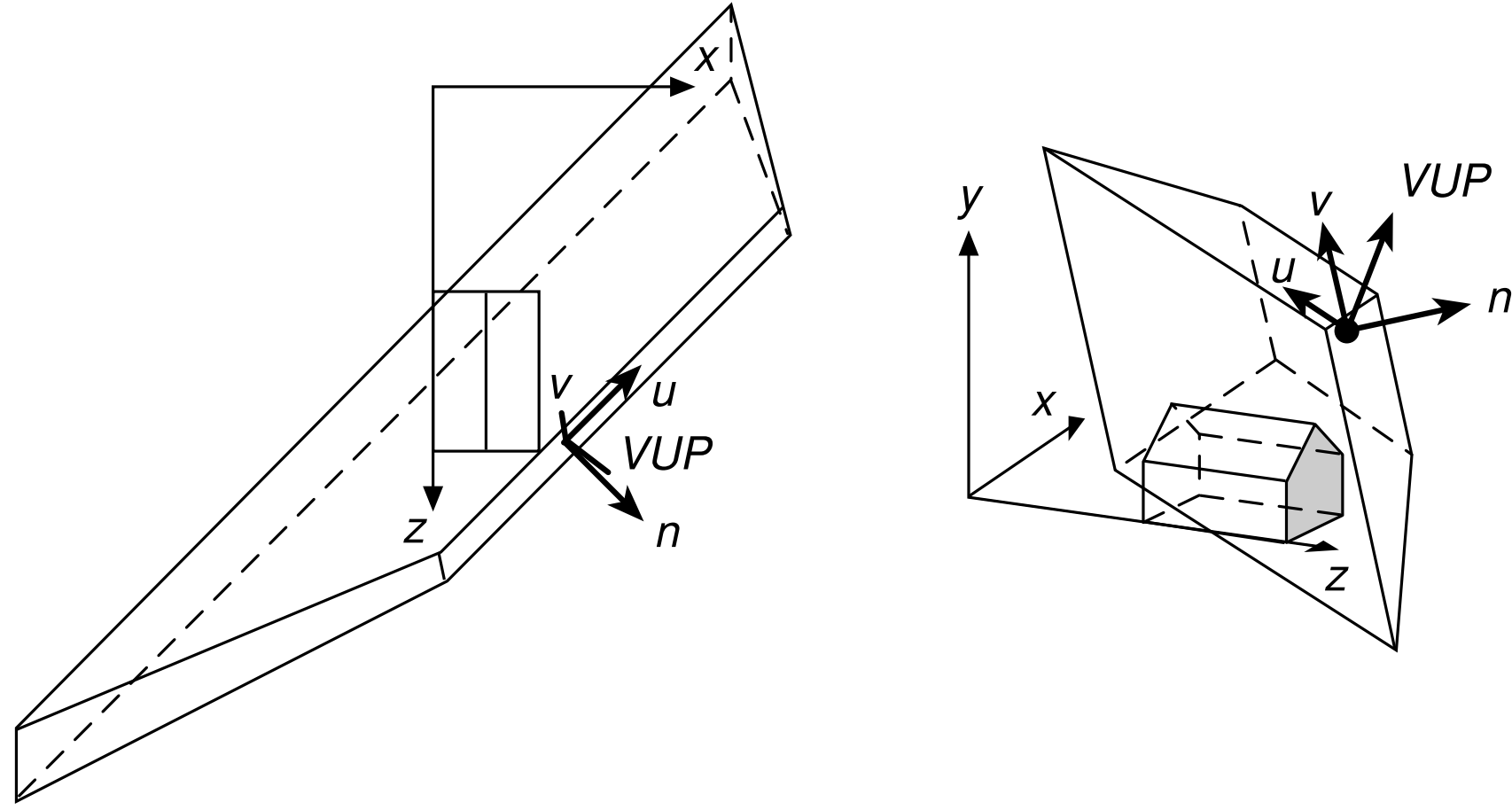
Viewing Transformations



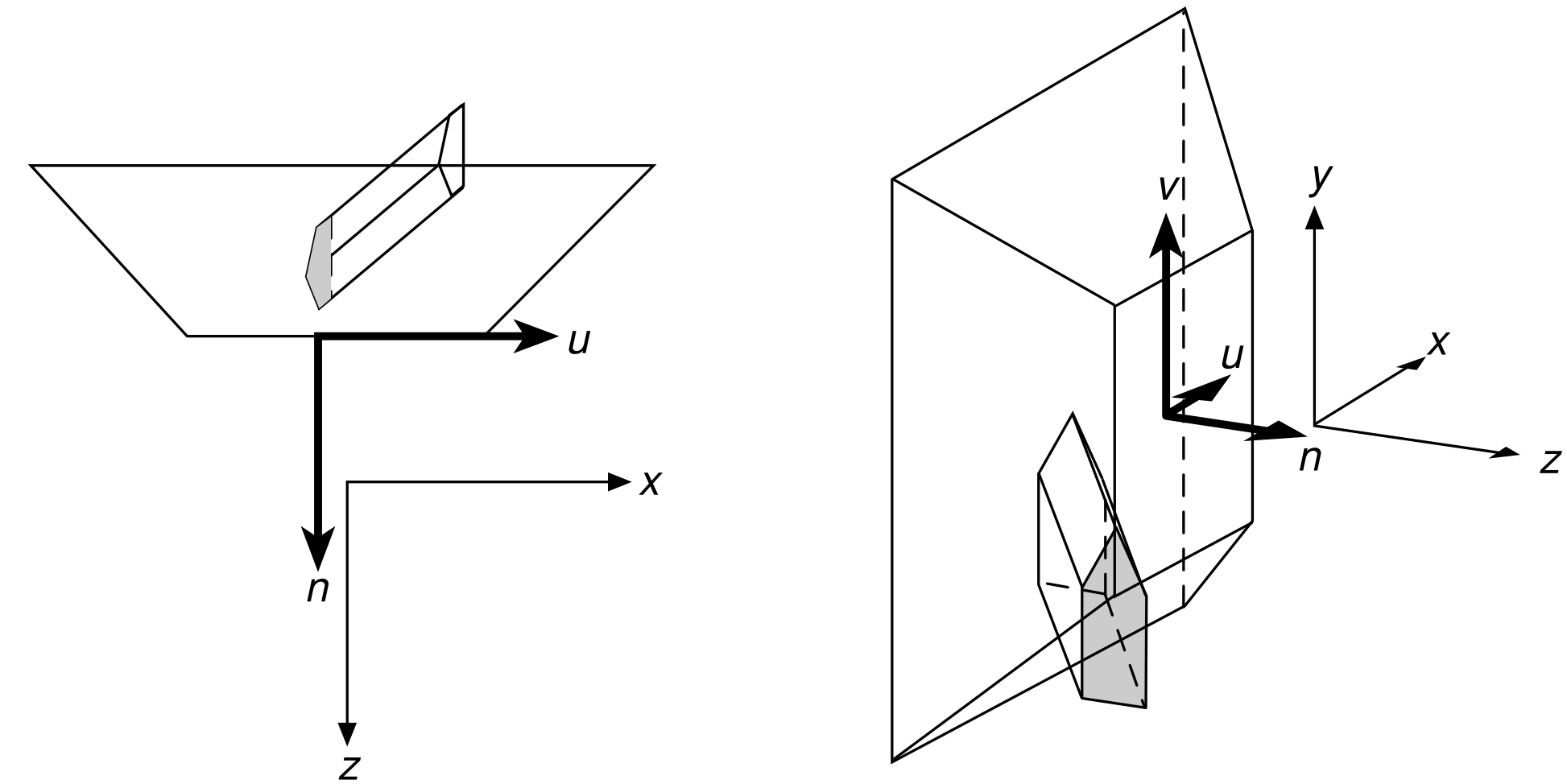
© [A00] Figure 7.03

- Basic transform-clip-transform sequence
 - ▶ combine projection and viewport-transform into one
- Find normalizing transformations N_{per} (N_{par}) to transform arbitrary perspective (or parallel) view volume into canonical form
 - ▶ efficient clipping at the expense of normalizing invisible points

Normalized Perspective Projection



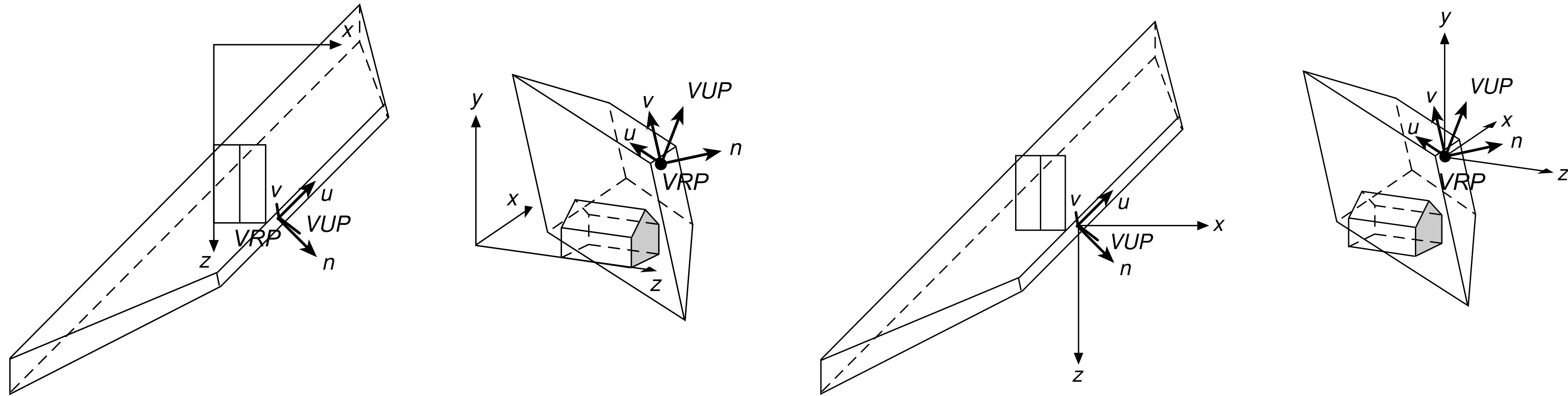
© [AW94] Figure 6.41



- N_{per} is a composition of

1. Translation of VRP to origin
2. Rotation of VRC to align with principal axes
3. Translation of the center of projection PRP to the origin
4. Shear such that the center line of the view volume becomes the z-axis
5. Scale such that the view volume becomes the canonical perspective view volume

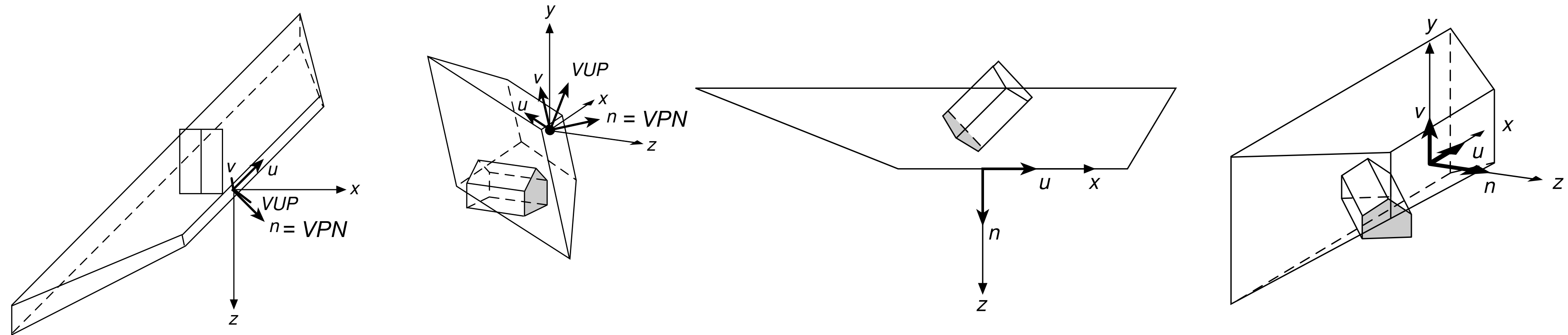
World-to-VRC 1



© [AW94] Figure 6.41

- Changing origin to be VRP is a simple translation defined by $T(-VRP)$
 - ▶ since the VRP is the center of our VRC system and given in the current world coordinate (WC) system

World-to-VRC 2



© [AW94] Figure 6.41

- Rotation R of the x, y, z axis into the u, v, n directions of the VRC-system is defined by the VPN and VUP vectors
 - ▶ exploit special orthogonal (rotation) matrices

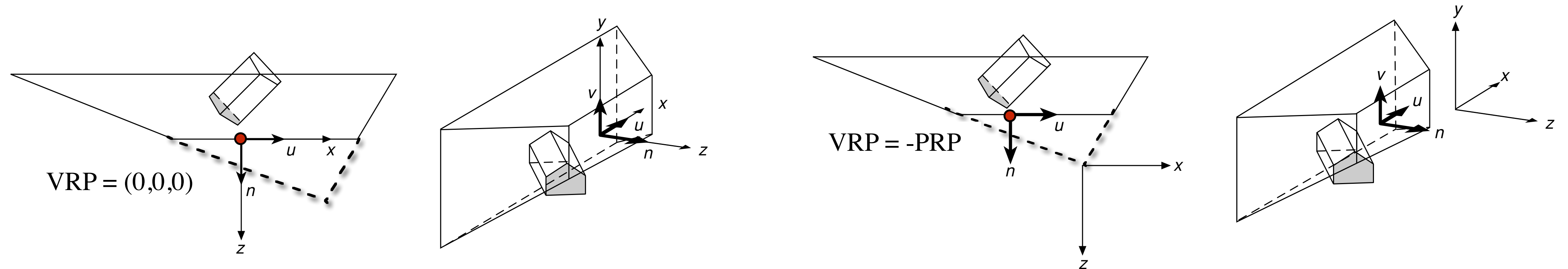
$$n = \frac{VPN}{\|VPN\|}$$

$$u = \frac{VUP \times n}{\|VUP \times n\|}$$

$$v = w \times u$$

$$R = \begin{bmatrix} u^T & 0 \\ v^T & 0 \\ n^T & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

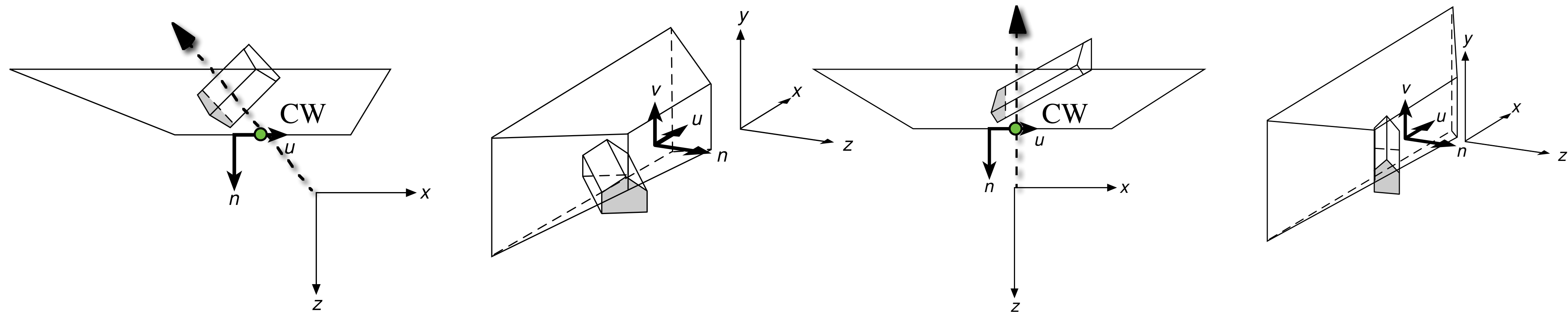
Make COP Origin



© [AW94] Figure 6.41

- Changing origin to be the center of projection is a simple translation by $T(-PRP)$
 - ▶ since PRP is already given in the current VRC system
- Moves the VRP from the origin to -PRP

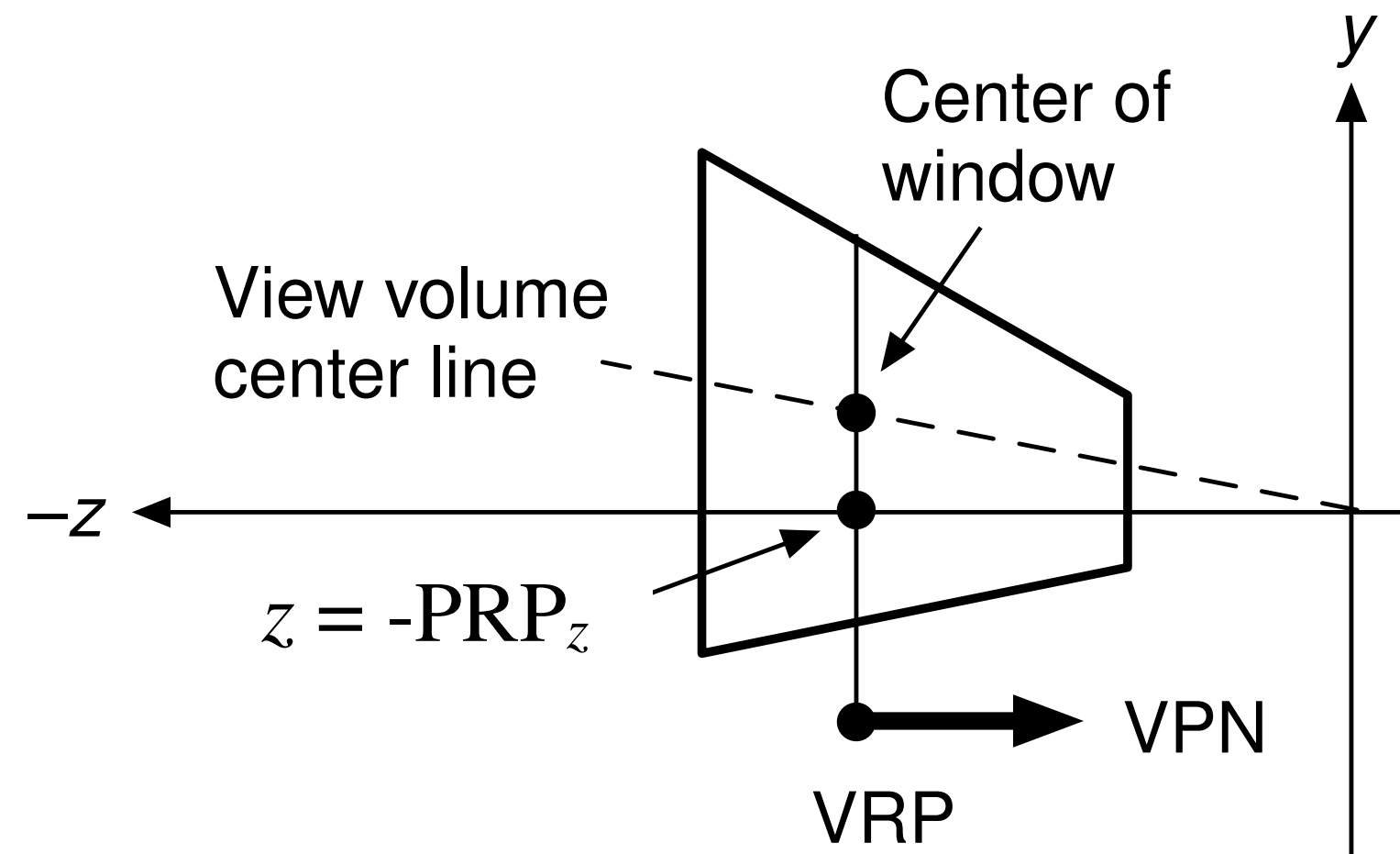
Shear Center



© [AW94] Figure 6.41

- This operation is necessary to align the center of the (clip) window CW with the z -axis
 - ▶ general input configuration may be a sheared view-frustum
- As points are moved in parallel to the x,y -plane this is a xy -shear transform

Shear Center



© [AW94] Figure 6.40

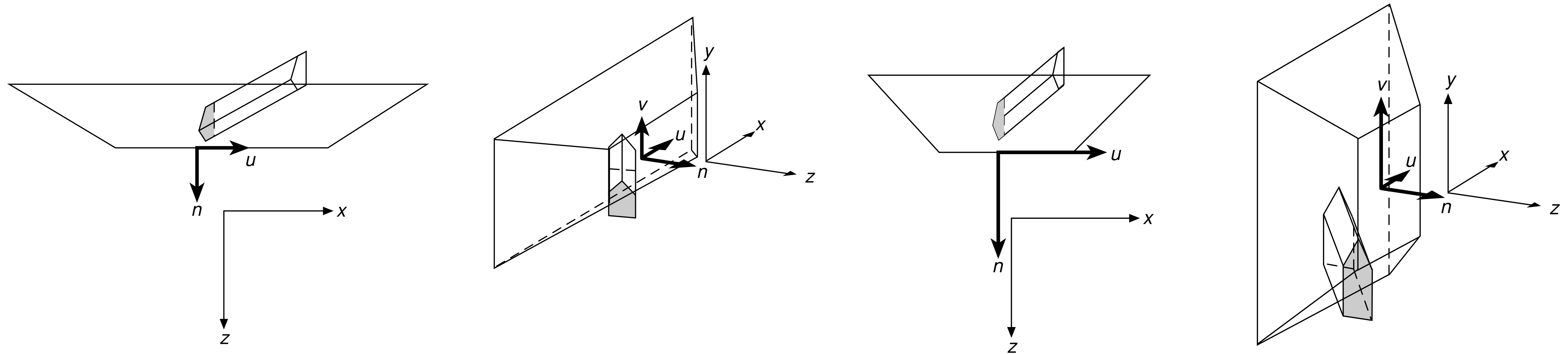
$$CW = \begin{bmatrix} \frac{u_{\min} + u_{\max}}{2} \\ \frac{v_{\min} + v_{\max}}{2} \\ -PRP_z \\ 1 \end{bmatrix}$$

$$SH_{\text{par}} = SH_{\text{per}} = \begin{bmatrix} 1 & 0 & \frac{-CW_x}{CW_z} & 0 \\ 0 & 1 & \frac{-CW_y}{CW_z} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Center of window (CW) must align with the the z -axis, thus x and y must become 0
 - ▶ CW defined by the center of the clip region $[u_{\min}, u_{\max}] \times [v_{\min}, v_{\max}]$ on the view plane and the current z -coordinate of the VRP (as transformed so far), but it holds that $VPR_z = -PPR_z$
 - ▶ $CW_x = 0.5 (u_{\min} + u_{\max})$ $CW_y = 0.5 (v_{\min} + v_{\max})$ $CW_z = VPR_z = -PPR_z$
- Shear $SH_{\text{par/per}}$ must transform the CW_x and CW_y coordinates of the CW point such that

$$CW_x + sh_x \cdot CW_z = 0 \text{ and } CW_y + sh_y \cdot CW_z = 0$$
 - ▶ thus $sh_x = -CW_x/CW_z$ $sh_y = -CW_y/CW_z$

Normalize Volume



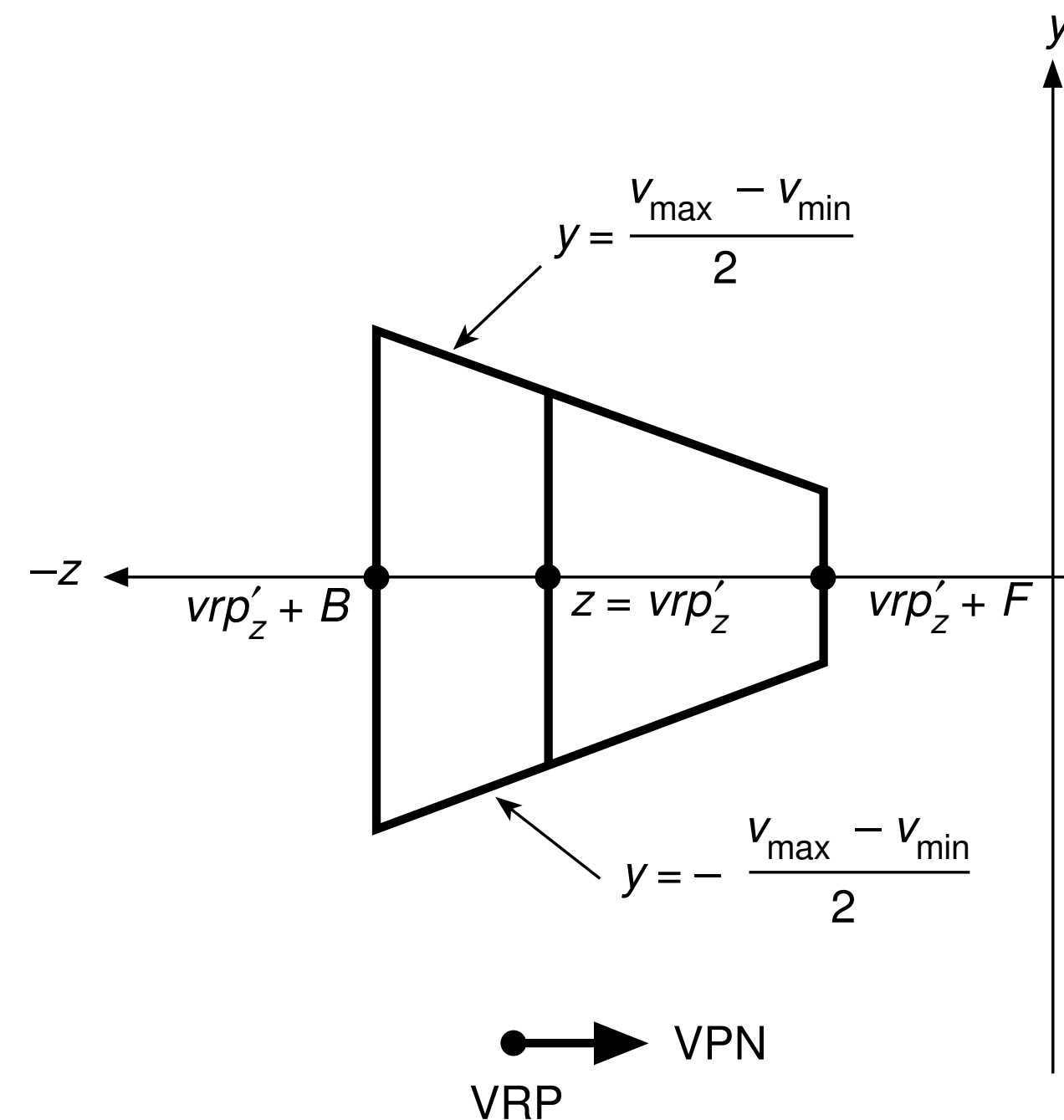
© [AW94] Figure 6.41

- Final step scales the view frustum to the canonical view volume
 - ▶ that fits into a unit half-cube bounded by $[-1,1] \times [-1,1] \times [-1,0]$
- Non-uniform scaling to unit-slopes in x and y followed by uniform scaling to move the back clipping plane to map to $z = -1$

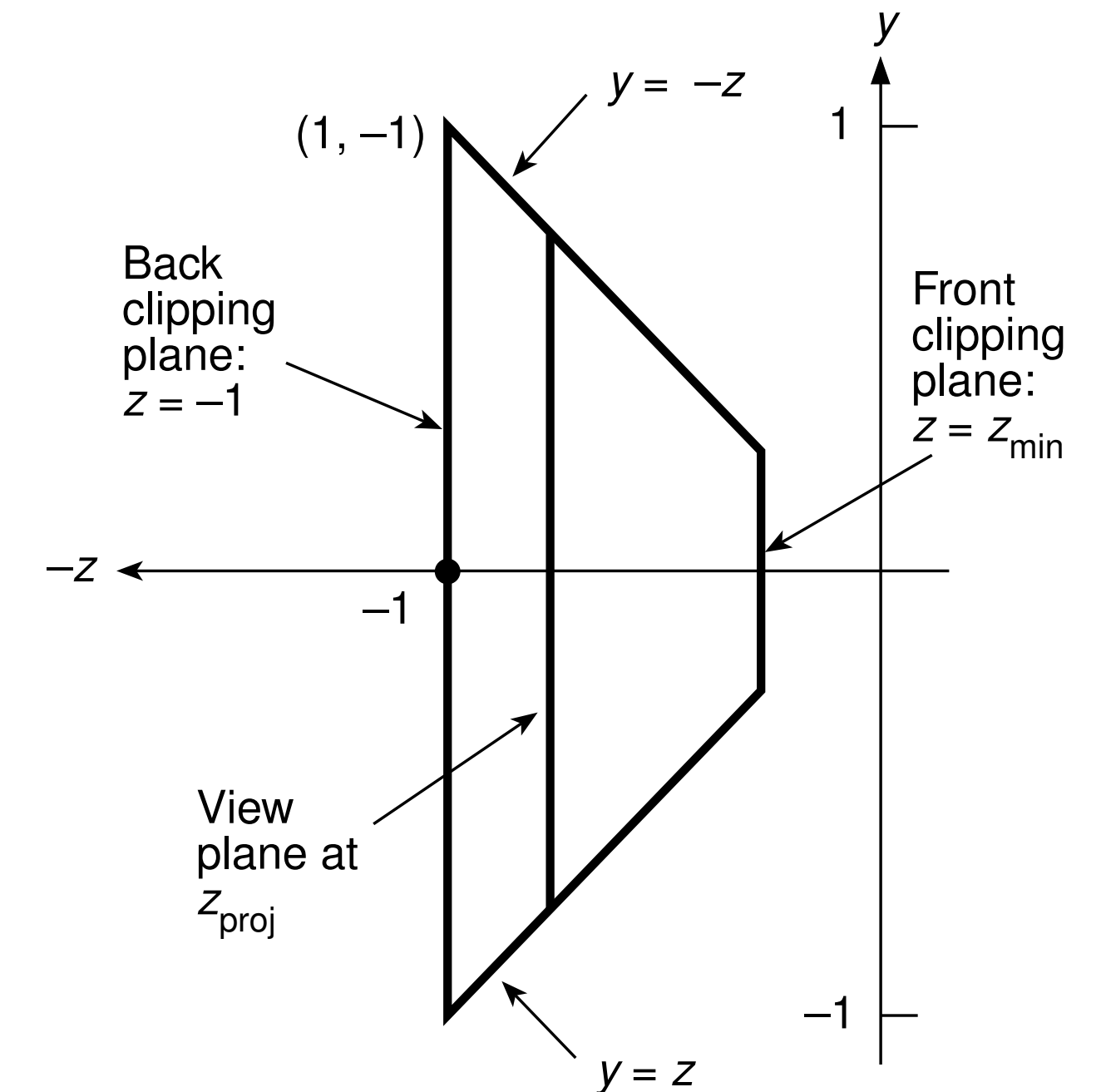
Normalize Volume 1

$$s_x = \frac{-2 \cdot \text{VRP}_z}{u_{\max} - u_{\min}}$$

$$s_y = \frac{-2 \cdot \text{VRP}_z}{v_{\max} - v_{\min}}$$



(a) Before scaling



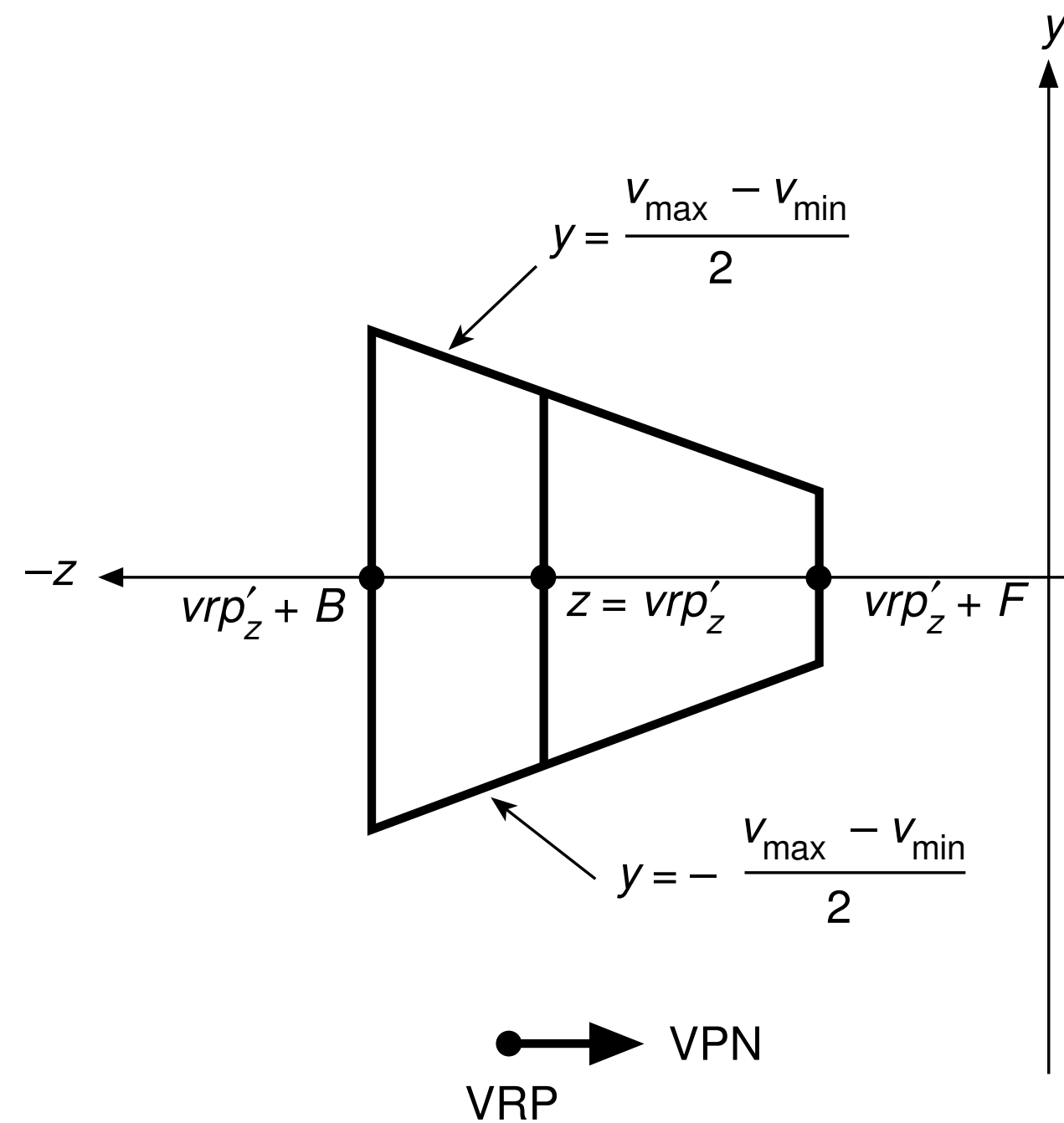
(b) After scaling

© [AW94] Figure 6.42

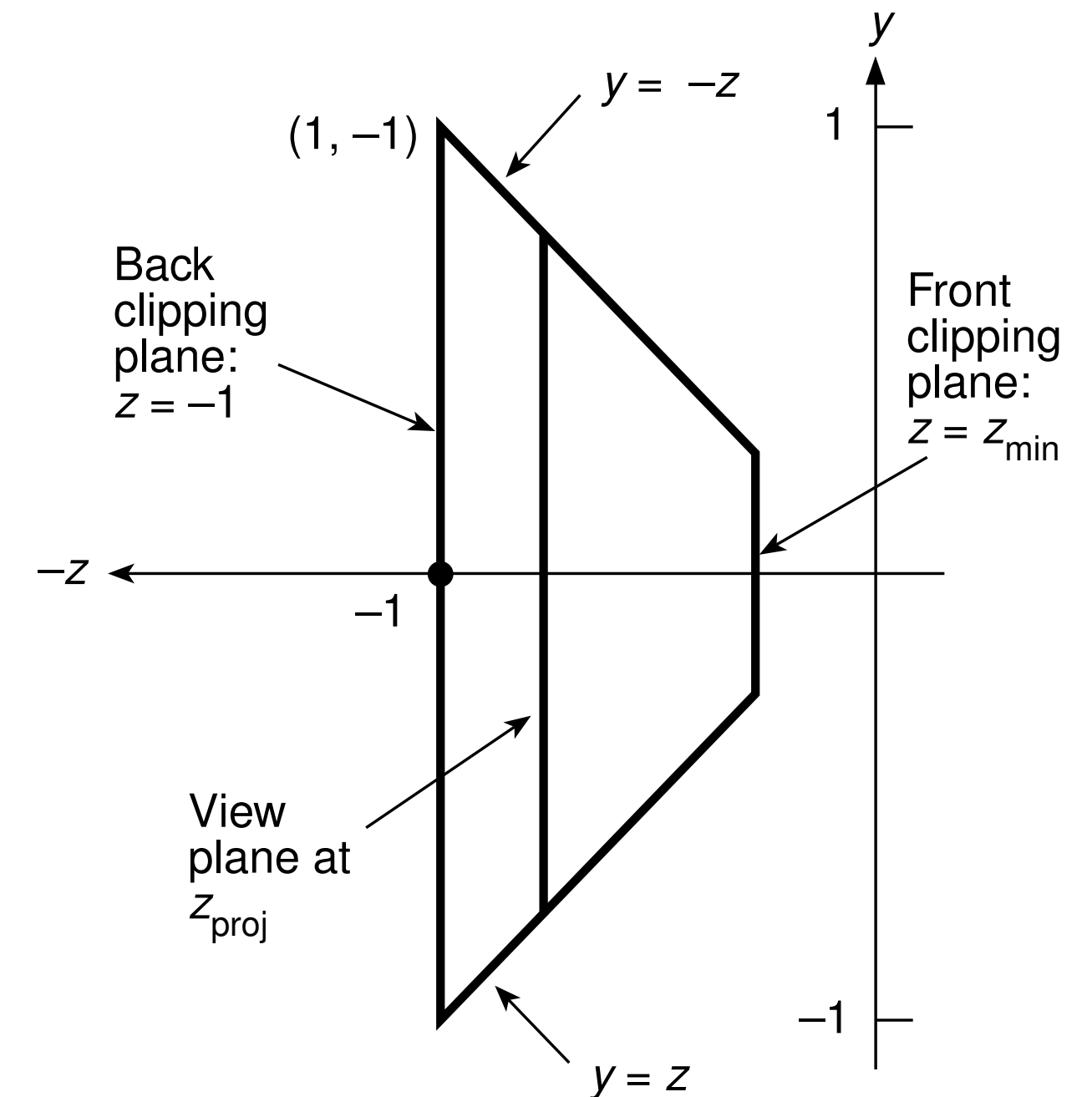
- Scale/stretch by s_{xy} such that the x and y coordinates of the corners of the clipping window exactly become $\pm \text{VRP}_z$ (but note that $\text{VRP}_z = -\text{PRP}_z$)
 - ▶ the x and y -coordinates of the CW corners before scaling are: $\pm 0.5 (u_{\max} - u_{\min})$ and $\pm 0.5 (v_{\max} - v_{\min})$
 - ▶ use the current z -coordinate of the VRP (as transformed so far by all previous steps)
 - ▶ corresponding scale factors are thus $s_x = -2 \text{VRP}_z / (u_{\max} - u_{\min})$ and $s_y = -2 \text{VRP}_z / (v_{\max} - v_{\min})$

Normalize Volume 2

$$s = \frac{-1}{\text{VRP}_z + B} \quad S_{\text{per}} = \begin{bmatrix} s_x \cdot s & 0 & 0 & 0 \\ 0 & s_y \cdot s & 0 & 0 \\ 0 & 0 & s & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



(a) Before scaling



(b) After scaling

© [AW94] Figure 6.42

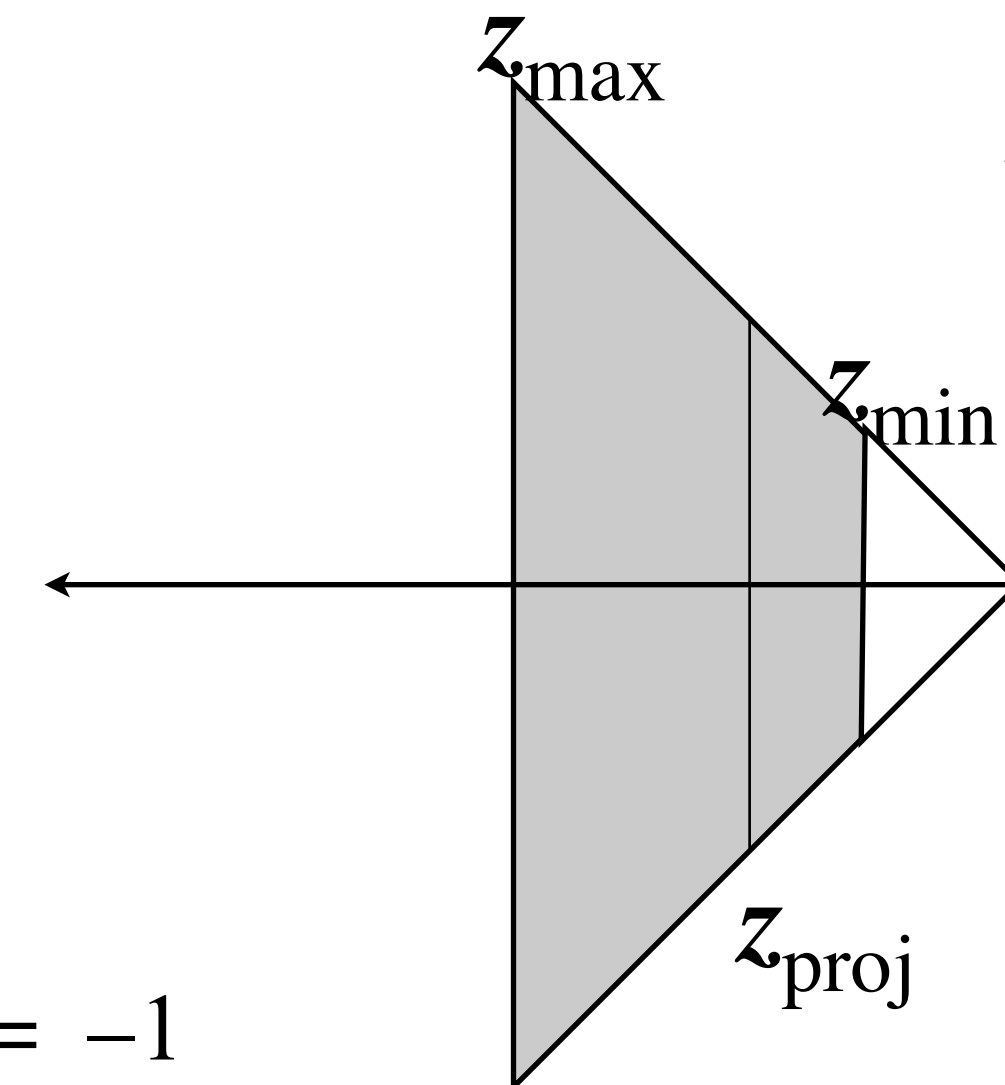
- Back clipping plane is at distance $z = \text{VRP}_z + B$, thus scaling S_z must change $\text{VRP}_z + B$ to be -1.0
 - ▶ use of current z -coordinate of the VRP (as transformed so far by all previous steps)
 - ▶ corresponding uniform scaling factor is thus $s = -1 / (\text{VRP}_z + B)$
- Final scaling matrix S_{per} is formed from the x , y and global scaling factors s_x , s_y and s

Normalized Perspective Transformation N_{per}

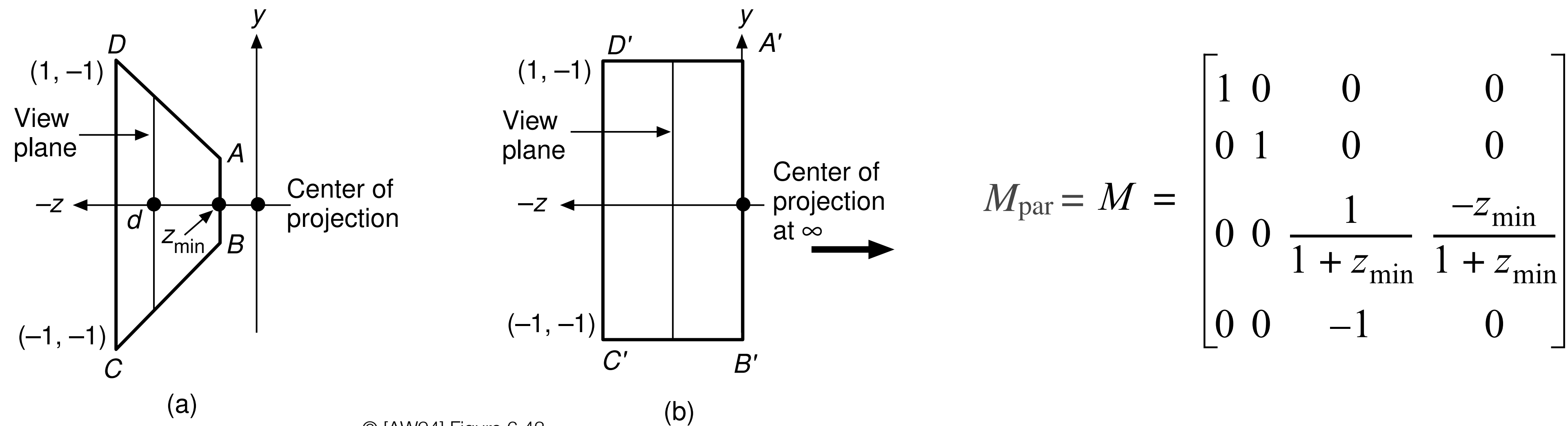
$$N_{\text{per}} = S_{\text{per}} \cdot SH_{\text{per/par}} \cdot T(-\text{PRP}) \cdot R \cdot T(-\text{VRP})$$

- N_{per} fully given by the developed transformation sequence
- The normalized perspective transform also modifies the positions of view- and clipping-planes along the z -axis

$$z_{\text{proj}} = \frac{\text{VRP}_z}{\text{VRP}_z + B} \quad z_{\text{min}} = -\frac{\text{VRP}_z + F}{\text{VRP}_z + B} \quad z_{\text{max}} = -\frac{\text{VRP}_z + B}{\text{VRP}_z + B} = -1$$



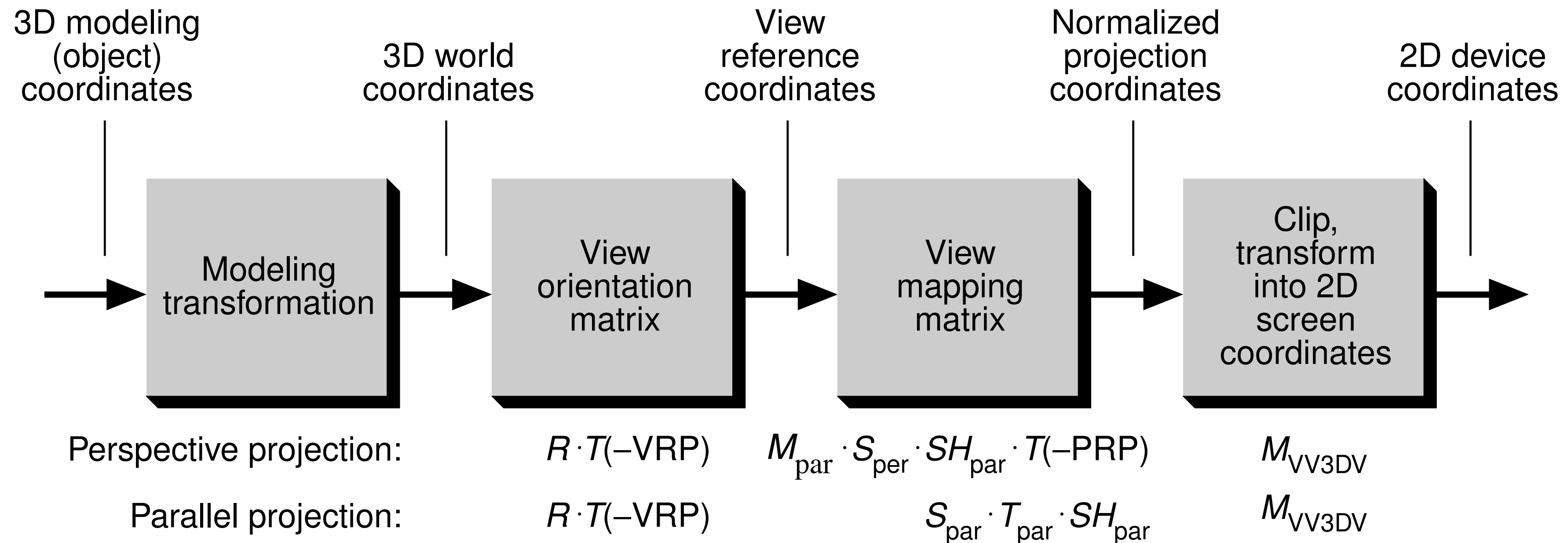
Conversion to Parallel-Projection Canonical View Volume



© [AW94] Figure 6.43

- Parallel-projection canonical view volume transform M_{par} further simplifies clipping and display
 - ▶ standard unit half-cube clipping rectangle $[-1, 1] \times [-1, 1] \times [-1, 0]$
 - ▶ transformation M_{par} can be combined with N_{per}
 - ▶ final screen-projection achieved simply by omitting z -coordinate

Coordinate Transformations



© [AW94] Figure 6.46

- Applied transformations define the progression of coordinate systems

$$M_{\text{VV3DV}} \cdot M_{\text{par}} \cdot S_{\text{per}} \cdot SH_{\text{per/par}} \cdot T(-PRP) \cdot R \cdot T(-VRP)$$

- ▶ model - world - view - normalized projection - viewport
- ▶ M_{par} being the conversion to the parallel-projection canonical view volume

Look-at User Navigation

- **Move the camera viewpoint** to a particular **location** $P (p_x, p_y, p_z)$ and **look into** a given **direction** $v(v_x, v_y, v_z)$
 - ▶ standard navigation task in interactive environments
- Typically graphics APIs have fixed canonical view settings:
 - ➔ **e.g. viewpoint at origin, view direction along one specific axis**

Virtual camera is always at origin and model-view object transformation must achieve desired camera placement.

Camera Placement

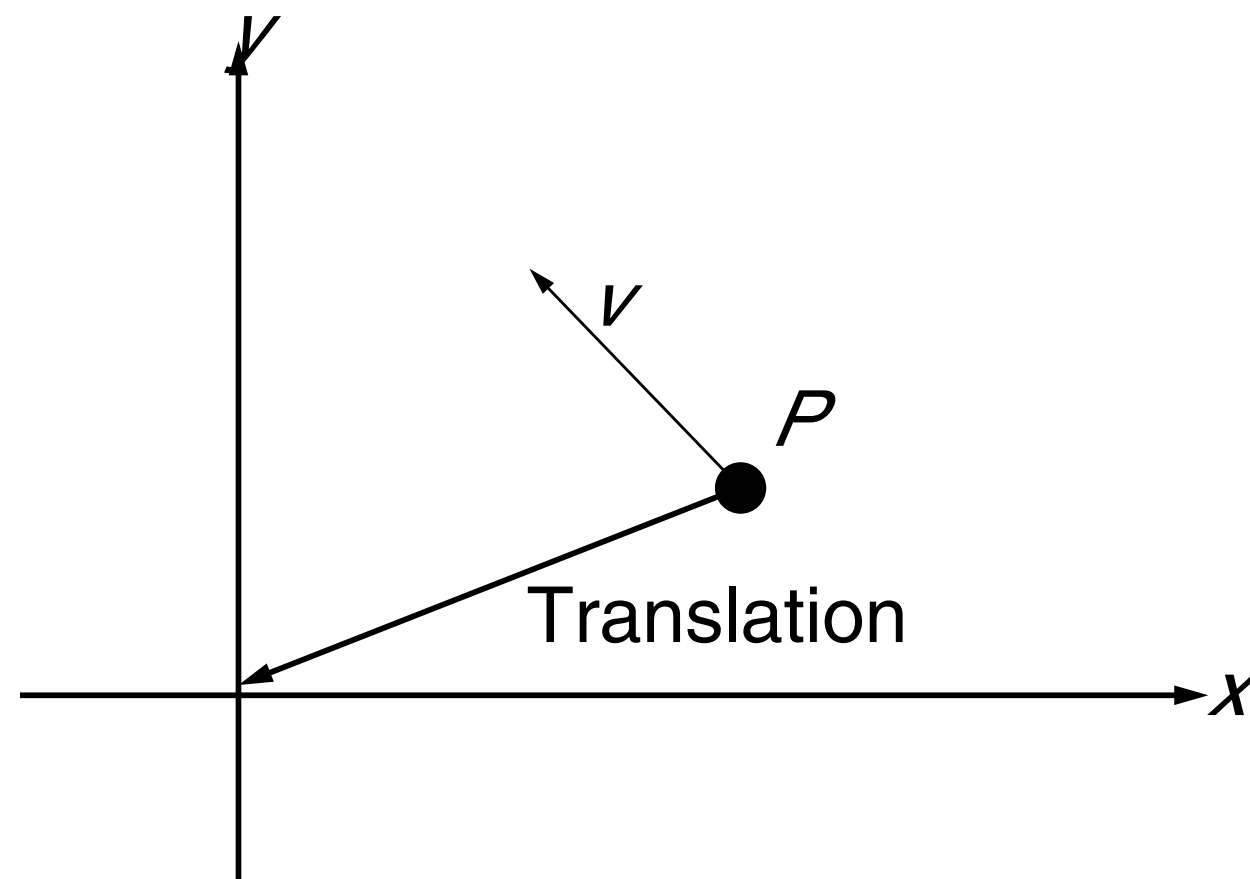
- Any view from location P in the direction of v can be achieved by:

1. change coordinate system such that P is the origin, canonical camera position
2. rotate principal axis such that v aligns with the canonical camera viewing axis

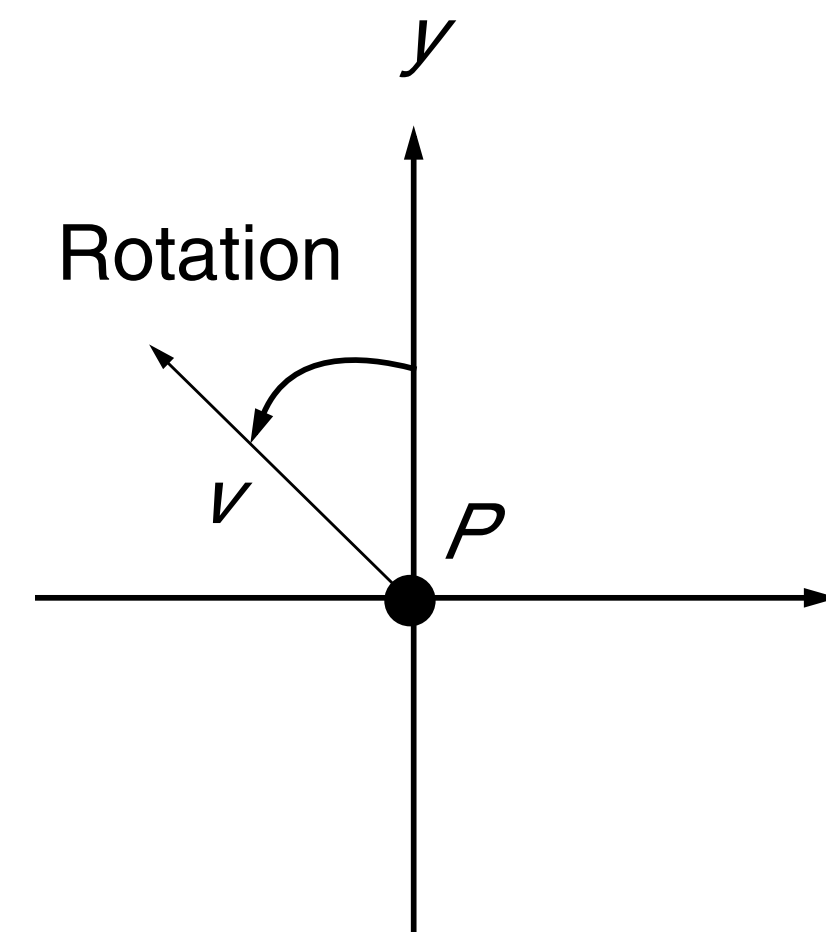
- Transform:

$$M = R \cdot T \text{ with } T = (-p_x, -p_y, -p_z)$$

- ▶ R must rotate coordinate system axis such that v is aligned with the y -axis



Example in 2D, viewing along y -axis



Camera Rotation

- Exploit property of **special orthogonal** matrices

In 2D, **viewing along y-axis**:

- ▶ $v=(v_x, v_y)$ can be used as 2nd row vector of R
- ▶ 1st row vector defined by vector perpendicular to v

In 3D, **viewing along z-axis**:

- ▶ $v=(v_x, v_y, v_z)$ can be used as 3rd row vector of R
- ▶ horizontal, x -axis can be defined perpendicular to v and 'old' y -axis using the cross-product
 $(0, 1, 0) \times v = (v_z, 0, -v_x)$ as 1st row vector
- ▶ remaining 2nd row vector must be perpendicular to the other two, given by cross-product
 $(v_x, v_y, v_z) \times (v_z, 0, -v_x) = (-v_x \cdot v_y, v_z^2 + v_x^2, -v_y \cdot v_z)$

- Note that v and all derived row vectors must be unit length!

$$R = \begin{bmatrix} v_y & -v_x & 0 \\ v_x & v_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$R = \begin{bmatrix} v_z & 0 & -v_x & 0 \\ -v_x \cdot v_y & v_z^2 + v_x^2 & -v_y \cdot v_z & 0 \\ v_x & v_y & v_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Copyrights

- Many figures in these slides are copyright protected as indicated. Text and all other figures are copyright protected by Renato Pajarola.
- [A06] Electronic Book Support for Interactive Computer Graphics: A Top-Down Approach Using OpenGL by Edward Angel. Copyright E. Angel 2006. (http://www.cs.unm.edu/~angel/BOOK/INTERACTIVE_COMPUTER_GRAPHICS/FOURTH_EDITION/)
- [A00] Electronic Book Support for Interactive Computer Graphics: A Top-Down Approach Using OpenGL by Edward Angel. Copyright E. Angel 2000. (http://www.cs.unm.edu/~angel/BOOK/INTERACTIVE_COMPUTER_GRAPHICS/SECOND_EDITION/)
- [AW94] Electronic Instructors Manual (EIM) for Introduction to Computer Graphics by James Foley, Andries van Dam, Steven Feiner, John Hughes, and Richard Phillips. Copyright Addison Wesley 1994.
- [AC97] Learning Rendering CD accompanying 3D Computer Graphics by Alan Watt. Copyright A. Watt and L. Cooper 1997. (Permission for copying and distribution without direct commercial advantage. To copy otherwise, or to republish, requires specific permission.)
- [S02] Electronic Figures for Fundamentals of Computer Graphics by Peter Shirley. Copyright P. Shirley 2002. (<http://www.cs.utah.edu/~shirley/fcg/figures/>)